Programming Concepts - Memory Tricks & Mnemonics

Easy-to-remember tricks for mastering programming concepts!

SOLID Principles

Memory Trick: "SOLID" itself is the mnemonic!

S - Single Responsibility Principle

- A class should have only ONE reason to change
- One class = One job
- Example: `UserRepository` only handles database operations, not email sending

O - Open/Closed Principle

- Open for extension, Closed for modification
- Add new features without changing existing code
- Example: Use interfaces/abstract classes to extend behavior

L - Liskov Substitution Principle

- Subtypes must be substitutable for their base types
- Child classes should work wherever parent class works
- Example: If `Bird` has `fly()`, don't make `Penguin` extend `Bird`

I - Interface Segregation Principle

- Many specific interfaces are better than one general interface
- Don't force classes to implement methods they don't use
- Example: Split `Worker` into `Workable`, `Eatable`, `Sleepable`

D - Dependency Inversion Principle

- Depend on abstractions, not concrete implementations
- High-level modules shouldn't depend on low-level modules
- Example: Use `PaymentProcessor` interface instead of `StripePayment` class directly

Alternative Memory Trick:

"Some Old Ladies In Denmark"

- **S**ingle Responsibility
- **O**pen/Closed
- **L**iskov Substitution
- **I**nterface Segregation
- **D**ependency Inversion

OOP Concepts (4 Pillars)

Memory Trick: "A PIE" or "APIE"

A - Abstraction

- Hide complex implementation details

- Show only essential features
- Example: You drive a car without knowing engine internals

P - **Polymorphism**

- Many forms of the same thing
- Same method name, different behavior
- Example: `draw()` method works differently for Circle, Square, Triangle

I - **Inheritance**

- Child class inherits properties from parent class
- Code reusability
- Example: `Dog` and `Cat` inherit from `Animal`

E - **Encapsulation**

- Bundle data and methods together
- Hide internal state (private fields)
- Example: Private variables with public getters/setters

Alternative Memory Trick:

"All Programmers In Europe"

- **A**bstraction
- **P**olymorphism
- **I**nheritance
- **E**ncapsulation

SQL Query Execution Order

Memory Trick: "Some Frogs Will Go Hopping Over Logs"

S - **SELECT** - What columns to retrieve **F** - **FROM** - Which table(s) **W** - **WHERE** - Filter rows before grouping **G** - **GROUP BY** - Group rows **H** - **HAVING** - Filter groups after aggregation **O** - **ORDER BY** - Sort results **L** - **LIMIT** - Limit number of results

---

HTTP Methods (REST API)

Memory Trick: "Good Programmers Prefer Deleting Poorly Optimized Code"

G - **GET** - Retrieve/Read data **P** - **POST** - Create new resource **P** - **PUT** - Update/Replace entire resource **D** - **DELETE** - Remove resource **P** - **PATCH** - Partial update **O** - **OPTIONS** - Get supported methods **C** - **CONNECT** - Establish tunnel

Simplified Version: "GPS"

G - **GET** - Read **P** - **POST** - Create **P** - **PUT** - Update **D** - **DELETE** - Delete

---

Design Patterns Categories

Memory Trick: "CBS News"

C - **Creational Patterns**

- How objects are created
- Examples: Singleton, Factory, Builder, Prototype

B - **Behavioral Patterns**

- How objects interact and communicate
- Examples: Observer, Strategy, Command, Iterator

S - **Structural Patterns**

- How objects are composed
- Examples: Adapter, Decorator, Facade, Proxy

Common Creational Design Patterns

Memory Trick: "Some Factories Build Awesome Products"

S - **Singleton** - Only one instance exists **F** - **Factory** - Creates objects without specifying exact class **B** - **Builder** - Constructs complex objects step by step **A** - **Abstract Factory** - Creates families of related objects **P** - **Prototype** - Creates objects by cloning existing ones

---

Exception Handling Order (Try-Catch-Finally)

Memory Trick: "Try Catching Fish"

T - **Try** - Code that might throw exception **C** - **Catch** - Handle the exception **F** - **Finally** - Always executes (cleanup code)

```
try {
    // risky code
} catch (Exception e) {
    // handle error
} finally {
    // cleanup (always runs)
}
```

ACID Properties (Database Transactions)

Memory Trick: "A Car Is Durable"

A - **Atomicity** - All or nothing (transaction completes fully or not at all) **C** - **Consistency** - Database remains in valid state **I** - **Isolation** - Transactions don't interfere with each other **D** - **Durability** - Committed data is permanent

---

CAP Theorem (Distributed Systems)

Memory Trick: "CAP" (You can only pick 2 out of 3)

C - **Consistency** - All nodes see same data at same time **A** - **Availability** - System always responds to requests **P** - **Partition Tolerance** - System works despite network failures

Rule: You can only guarantee 2 out of 3!

- **CP**: Consistency + Partition Tolerance (e.g., MongoDB, HBase)
- **AP**: Availability + Partition Tolerance (e.g., Cassandra, DynamoDB)
- **CA**: Consistency + Availability (e.g., Traditional RDBMS - but rare in distributed systems)

Microservices Patterns

Memory Trick: "SAGA Circuits Break Easily"

S - **SAGA Pattern** - Distributed transactions across services **A** - **API Gateway** - Single entry point for all clients **G** - **Gateway Aggregation** - Combine multiple service calls **A** - **API Composition** - Aggregate data from multiple services

C - **Circuit Breaker** - Prevent cascading failures **B** - **Bulkhead** - Isolate resources to prevent total failure **E** - **Event Sourcing** - Store state changes as events

---

Git Workflow Commands

Memory Trick: "All Coders Push Merges"

A - **Add** - Stage changes (`git add`) **C** - **Commit** - Save changes locally (`git commit`) **P** - **Push** - Upload to remote (`git push`) **M** - **Merge** - Combine branches (`git merge`)

Extended Version: "All Coders Carefully Push Merges Properly"

A - **Add** - `git add .` **C** - **Commit** - `git commit -m "message"` **C** - **Check status** - `git status` **P** - **Pull** - `git pull` (get latest changes) **P** - **Push** - `git push` **M** - **Merge** - `git merge branch-name`

Data Structures Performance

Memory Trick: "Arrays Are Fast, Lists Link Slowly"

Arrays:

- **Fast** access by index: O(1)
- **Slow** insertion/deletion: O(n)

Linked Lists:

- **Slow** access by index: O(n)
- **Fast** insertion/deletion at ends: O(1)

Hash Maps:

- **Fast** everything (average): O(1)
- Search, Insert, Delete

Trees (BST):

- **Balanced** operations: O(log n)
- Search, Insert, Delete

Testing Pyramid

Memory Trick: "Unit Integration End-to-end" (UIE - "You I E")

From bottom (most) to top (least):

U - **Unit Tests** (70%)

- Test individual functions/methods
- Fast, isolated, many tests

I - **Integration Tests** (20%)

- Test multiple components together
- Medium speed, moderate number

E - **End-to-End Tests** (10%)

- Test entire application flow
- Slow, expensive, few tests

Spring Boot Annotations

Memory Trick: "REST Controllers Serve Requests Automatically"

@RestController - Marks class as REST API controller **@Controller** - Marks class as MVC controller **@Service** - Business logic layer **@Repository** - Data access layer **@Autowired** - Automatic dependency injection

Layered Architecture: "CRS"

C - **@Controller** - Presentation layer **R** - **@Repository** - Data layer **S** - **@Service** - Business layer

---

Java Collections Hierarchy

Memory Trick: "List Set Queue Map"

List - Ordered, allows duplicates

- ArrayList, LinkedList, Vector

Set - Unordered, no duplicates

- HashSet, TreeSet, LinkedHashSet

Queue - FIFO (First In First Out)

- PriorityQueue, LinkedList, ArrayDeque
- **FIFO**: First In First Out (like a line at a store)
- Example: First person in line gets served first

Stack - LIFO (Last In First Out)

- Stack class (legacy), Deque interface (preferred)
- **LIFO**: Last In First Out (like a stack of plates)
- Example: Last plate placed on top is the first one taken

Map - Key-Value pairs

- HashMap, TreeMap, LinkedHashMap

Memory Trick for FIFO vs LIFO:

"Queue = Line at Store (FIFO)"

- People join at back, leave from front
- First person in line → First person served

"Stack = Stack of Plates (LIFO)"

- Add plates on top, remove from top
- Last plate added → First plate removed

Visual Representation:

```
FIFO (Queue):
IN → [1][2][3][4] → OUT
     ↑           ↑
   (back)     (front)
   (last)     (first)

[1] = Entered FIRST  → Will leave FIRST  (at front)
[2] = Entered 2nd    → Will leave 2nd
[3] = Entered 3rd    → Will leave 3rd
[4] = Entered LAST   → Will leave LAST   (at back)

Add at back/rear (enqueue)
Remove from front (dequeue)

Example Timeline:
Step 1: queue.offer(1) → [1]           (1 entered first)
Step 2: queue.offer(2) → [1][2]        (2 entered second)
Step 3: queue.offer(3) → [1][2][3]     (3 entered third)
Step 4: queue.offer(4) → [1][2][3][4]  (4 entered last)
Step 5: queue.poll()   → [2][3][4]     (1 leaves first - FIFO!)

LIFO (Stack):
        ↑ OUT
        ↓ IN
       [4]  ← Entered LAST  → Leaves FIRST  (Last In, First Out)
       [3]  ← Entered 3rd   → Leaves 2nd
       [2]  ← Entered 2nd   → Leaves 3rd
       [1]  ← Entered FIRST → Leaves LAST   (First In, Last Out)

Add at top (push)
Remove from top (pop)

Example Timeline:
Step 1: stack.push(1) → [1]           (1 entered first, at bottom)
Step 2: stack.push(2) → [1][2]        (2 entered second)
Step 3: stack.push(3) → [1][2][3]     (3 entered third)
```

```
Step 4: stack.push(4) → [1][2][3][4]  (4 entered last, at top)
Step 5: stack.pop()   → [1][2][3]     (4 leaves first - LIFO!)
```

Code Examples:

```java
// Queue (FIFO)
Queue<Integer> queue = new LinkedList<>();
queue.offer(1);  // [1]
queue.offer(2);  // [1, 2]
queue.offer(3);  // [1, 2, 3]
queue.poll();    // Removes 1 → [2, 3]

// Stack (LIFO)
Stack<Integer> stack = new Stack<>();
stack.push(1);   // [1]
stack.push(2);   // [1, 2]
stack.push(3);   // [1, 2, 3]
stack.pop();     // Removes 3 → [1, 2]

// Deque (Double-Ended Queue) - can work as BOTH Queue and Stack
Deque<Integer> deque = new ArrayDeque<>();

// ============================================
// USING DEQUE AS QUEUE (FIFO)
// ============================================
// Rule: Add at BACK (Last), Remove from FRONT (First)

deque.offerLast(1);   // Add at end/back    → [1]
deque.offerLast(2);   // Add at end/back    → [1, 2]
deque.offerLast(3);   // Add at end/back    → [1, 2, 3]
deque.pollFirst();    // Remove from front  → Removes 1 → [2, 3]
deque.pollFirst();    // Remove from front  → Removes 2 → [3]

// Alternative Queue methods (same behavior):
deque.addLast(4);     // Same as offerLast  → [3, 4]
deque.removeFirst();  // Same as pollFirst  → Removes 3 → [4]

// ============================================
// USING DEQUE AS STACK (LIFO)
// ============================================
// Rule: Add at END (Last), Remove from END (Last)

Deque<Integer> stackDeque = new ArrayDeque<>();
stackDeque.offerLast(1);   // Push/Add at top    → [1]
stackDeque.offerLast(2);   // Push/Add at top    → [1, 2]
stackDeque.offerLast(3);   // Push/Add at top    → [1, 2, 3]
stackDeque.pollLast();     // Pop/Remove from top → Removes 3 → [1, 2]
stackDeque.pollLast();     // Pop/Remove from top → Removes 2 → [1]

// Alternative Stack methods (same behavior):
stackDeque.push(4);        // Same as offerFirst → [4, 1]
```

```java
stackDeque.pop();          // Same as pollFirst  → Removes 4 → [1]

// ============================================
// HOW TO CONFIRM WHICH MODE YOU'RE USING?
// ============================================

// METHOD 1: By Method Combination
// --------------------------------
// QUEUE (FIFO):  offerLast() + pollFirst()  OR  addLast() + removeFirst()
// STACK (LIFO):  offerLast() + pollLast()   OR  push() + pop()

// METHOD 2: Use Specific Interface Reference
// --------------------------------
// Force Queue behavior:
Queue<Integer> queueMode = new ArrayDeque<>();
queueMode.offer(1);  // Can only use Queue methods
queueMode.poll();    // Automatically FIFO

// Force Stack behavior (use Deque with stack methods):
Deque<Integer> stackMode = new ArrayDeque<>();
stackMode.push(1);   // Stack-like methods
stackMode.pop();     // Automatically LIFO

// METHOD 3: Consistent Method Naming Convention
// --------------------------------
// In your code, be consistent:

// For Queue - always use "Last" for add, "First" for remove:
public void useAsQueue() {
    Deque<Integer> queue = new ArrayDeque<>();
    queue.offerLast(1);   // Add
    queue.offerLast(2);   // Add
    queue.pollFirst();    // Remove (FIFO)
}

// For Stack - always use "Last" for both add and remove:
public void useAsStack() {
    Deque<Integer> stack = new ArrayDeque<>();
    stack.offerLast(1);   // Add (push)
    stack.offerLast(2);   // Add (push)
    stack.pollLast();     // Remove (pop) - LIFO
}

// Or use push/pop for clarity:
public void useAsStackClear() {
    Deque<Integer> stack = new ArrayDeque<>();
    stack.push(1);        // Push
    stack.push(2);        // Push
    stack.pop();          // Pop - LIFO
}
```

Deque Method Summary:

Operation	Queue (FIFO)	Stack (LIFO)
Add	`offerLast()` or `addLast()`	`offerLast()` or `push()`
Remove	`pollFirst()` or `removeFirst()`	`pollLast()` or `pop()`
Peek	`peekFirst()`	`peekLast()` or `peek()`

Key Takeaway:

The combination of methods determines the behavior:

- **Queue (FIFO)**: Add at one end (`Last`), Remove from other end (`First`)
- **Stack (LIFO)**: Add and Remove from the SAME end (`Last` or use `push` / `pop`)

Best Practice:

```
// ✅ CLEAR: Use Queue interface when you want FIFO
Queue<Integer> queue = new ArrayDeque<>();
queue.offer(1);
queue.poll();  // FIFO guaranteed

// ✅ CLEAR: Use Deque with push/pop when you want LIFO
Deque<Integer> stack = new ArrayDeque<>();
stack.push(1);
stack.pop();   // LIFO guaranteed

// ⚠️ CONFUSING: Using Deque without clear pattern
Deque<Integer> unclear = new ArrayDeque<>();
unclear.offerLast(1);
unclear.pollFirst();  // Is this queue or stack? Not immediately clear!
```

Kafka Components

Memory Trick: "Producers Consume Topics Partitioned By Brokers"

P - **Producer** - Sends messages **C** - **Consumer** - Reads messages **T** - **Topic** - Category/feed name **P** - **Partition** - Topic subdivisions for parallelism **B** - **Broker** - Kafka server that stores data

---

Docker Commands

Memory Trick: "Build Run Push Stop"

B - **Build** - Create image (`docker build`) **R** - **Run** - Start container (`docker run`) **P** - **Push** - Upload to registry (`docker push`) **S** - **Stop** - Stop container (`docker stop`)

Extended: "Build Images, Run Containers, Push Everywhere, Stop Carefully"

- **docker build** - Create image from Dockerfile
- **docker images** - List all images

- **docker run** - Create and start container
- **docker ps** - List running containers
- **docker push** - Push to Docker Hub
- **docker stop** - Stop running container

Agile Scrum Events

Memory Trick: "Sprint Planning Daily Review Retro"

S - **Sprint Planning** - Plan the sprint (start) **D** - **Daily Standup** - 15-min sync (every day) **R** - **Sprint Review** - Demo completed work (end) **R** - **Sprint Retrospective** - Reflect and improve (end)

REST API Status Codes

Memory Trick: "2 Success, 3 Redirect, 4 Client Error, 5 Server Error"

2xx - **Success**

- 200 OK, 201 Created, 204 No Content

3xx - **Redirection**

- 301 Moved Permanently, 302 Found, 304 Not Modified

4xx - **Client Error**

- 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found

5xx - **Server Error**

- 500 Internal Server Error, 502 Bad Gateway, 503 Service Unavailable

---

Clean Code Principles

Memory Trick: "KISS DRY YAGNI"

KISS - **Keep It Simple, Stupid**

- Simplicity is key
- Avoid unnecessary complexity

DRY - **Don't Repeat Yourself**

- Avoid code duplication
- Reuse code through functions/classes

YAGNI - **You Aren't Gonna Need It**

- Don't add functionality until needed
- Avoid over-engineering

Bonus: Programming Interview Topics

Memory Trick: "Data Structures And Algorithms Design Databases"

D - **Data Structures** - Arrays, Lists, Trees, Graphs **S** - **Sorting** - Quick, Merge, Bubble, Heap sort **A** - **Algorithms** - Search, Dynamic Programming, Greedy **A** - **Analysis** - Time/Space Complexity (Big O) **D** - **Design Patterns** - Singleton, Factory, Observer **D** - **Databases** - SQL, NoSQL, Indexing, Transactions

---

Tips for Creating Your Own Memory Tricks

1. **Use Acronyms** - First letter of each concept
2. **Create Stories** - Link concepts in a narrative
3. **Use Rhymes** - Make it catchy and memorable
4. **Visual Association** - Connect to images in your mind
5. **Personal Connection** - Relate to your own experiences

Remember: These tricks are learning aids. Understanding the concepts deeply is more important than just memorizing the mnemonics!